

TECHNICAL DOCUMENTATION

SkillPath.ai

AI-Powered Career Coaching & Resume Intelligence Platform

VERSION 2.0 • POST-MIGRATION

STATUS: PRODUCTION-READY MVP

Last Updated · 6 August 2026

CONTENTS

Table of Contents

1. Project Overview.....5

 Core Value Proposition.....5

 Target Audience.....5

2. Architecture6

 High-Level Architecture6

 Data Flow — Resume Analysis.....6

3. Tech Stack8

 Frontend8

 Backend8

 AI Providers.....9

4. Project Structure.....10

5. Database Schema.....12

 Dual Database Support.....12

 All 38 Tables.....12

6. Resume Parsing Pipeline.....14

 6.1 — File Upload & Validation.....14

 6.2 — Text Extraction14

 6.3 — Local Regex Parser (Tier 1)14

 6.4 — AI Provider Chain (Tier 2)17

 6.5 — Caching.....17

7. Scoring & Matching Algorithms18

 7.1 — Resume Score Breakdown.....18

 7.2 — Match Score19

 7.3 — Job Candidate Ranking19

 7.4 — Skill Gap Prioritization19

 7.5 — Field Prediction.....19

8. Authentication & Security.....20

 8.1 — JWT Token System.....20

 8.2 — Password Security20

 8.3 — OTP Verification20

 8.4 — Login Lockout21

 8.5 — Rate Limiting.....21

 8.6 — Security Headers21

 8.7 — CORS.....21

9. API Reference.....23

- Authentication Endpoints 23
- Analysis Endpoints 23
- Career Toolkit Endpoints 23
- Mock Interview Endpoints..... 24
- User Endpoints..... 25
- Sharing Endpoints..... 25
- Admin Endpoints 25
- Public Endpoints 26
- 10. Frontend Architecture 27
 - 10.1 — Technology Stack..... 27
 - 10.2 — Routing Structure 27
 - 10.3 — Authentication Flow 27
 - 10.4 — Key Components 28
 - 10.5 — State Management..... 28
- 11. Configuration & Environment..... 29
 - 11.1 — Required Environment Variables 29
 - 11.2 — Optional Environment Variables 29
 - 11.3 — Environment-Specific Behavior 30
- 12. Algorithms Deep Dive 31
 - 12.1 — Skill Inference Graph 31
 - 12.2 — Fuzzy Skill Matching 31
 - 12.3 — Date Parsing 32
 - 12.4 — Experience Block Parsing..... 33
 - 12.5 — Roadmap Generation 33
- 13. Testing..... 35
 - 13.1 — Test Suite Overview..... 35
 - 13.2 — Test Files..... 35
 - 13.3 — Test Environment 35
 - 13.4 — Key Test Cases 35
- 14. Deployment 37
 - 14.1 — Backend Deployment 37
 - 14.2 — Frontend Deployment..... 37
 - 14.3 — Environment Setup..... 37
 - 14.4 — Docker Support 37
- Appendix A: Skill Taxonomy..... 39
- Appendix B: Role Configurations 39

1. Project Overview

SkillPath is a full-stack SaaS platform that analyzes resumes using natural language processing and generative AI to provide career coaching, skill gap analysis, and personalized learning roadmaps. It combines a deterministic local parsing engine with an optional large-language-model layer so the product remains fully functional even without any AI provider configured.

Core Value Proposition

- **Resume Analysis** — Parse PDF/DOCX resumes and extract structured data
- **Skill Gap Analysis** — Compare candidate skills against role requirements
- **Career Coaching** — Generate personalized learning roadmaps
- **Job Matching** — Match candidates to market opportunities
- **Mock Interviews** — AI-powered interview practice

Target Audience

Audience	Use Case
Job Seekers	Improve resume, identify skill gaps, prepare for interviews
Career Changers	Plan transition path into tech roles
Recruiters	Evaluate candidate fit against job requirements
Hiring Managers	Understand market trends and skill demands

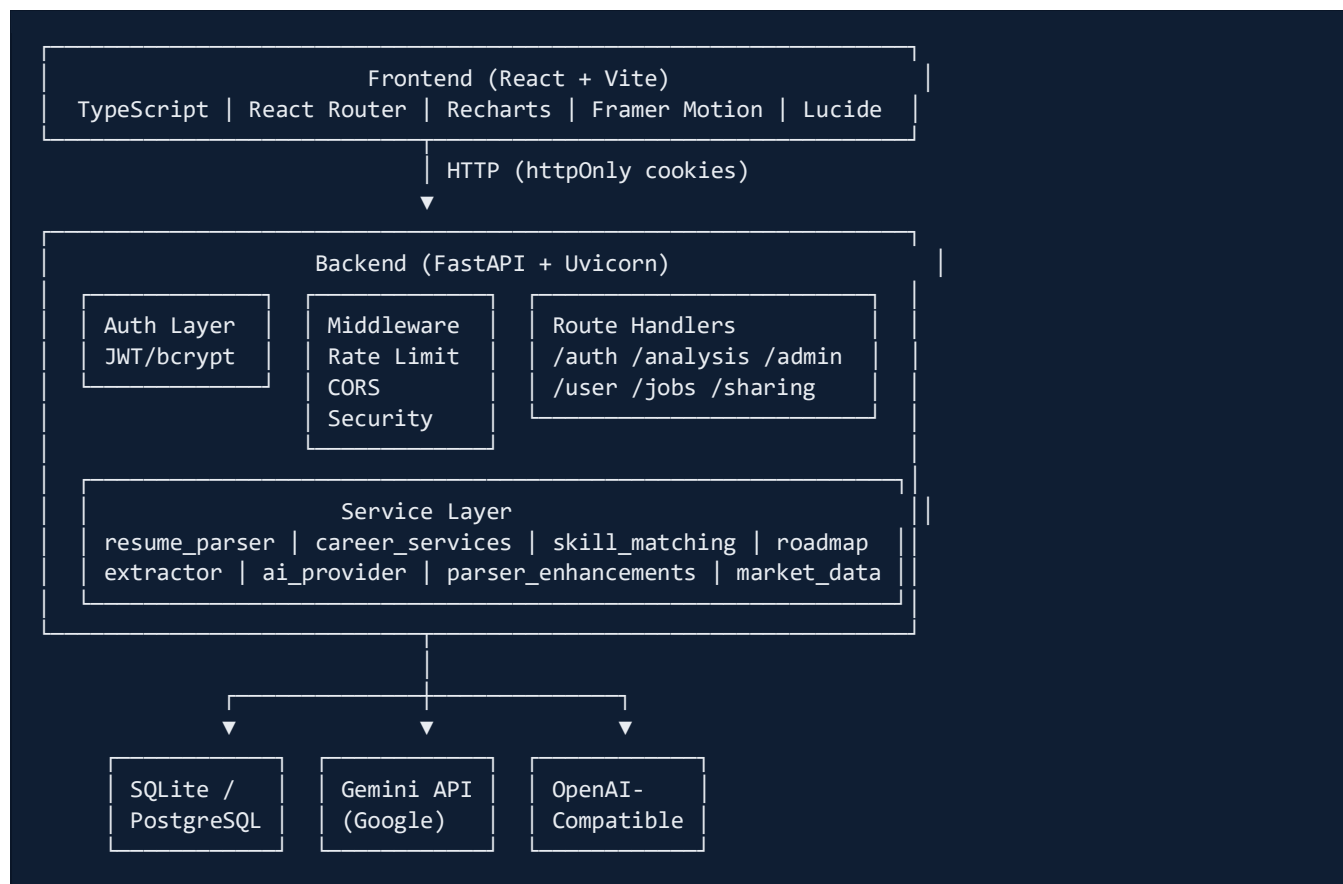
Why it matters

SkillPath's two-tier design (local regex/NLP parser + optional AI provider) means the product degrades gracefully: it always returns a usable analysis, and gets smarter automatically the moment an API key is added.

2. Architecture

High-Level Architecture

The system is split into three tiers: a React single-page frontend, a FastAPI backend organised into an auth layer, middleware stack and route handlers, and a service layer that performs the actual resume-analysis work. The service layer talks to a relational database and, optionally, to external AI providers.



Data Flow — Resume Analysis

Every uploaded resume passes through the same deterministic pipeline: validation, extraction, a two-tier parsing decision tree, score computation, and caching.





3. Tech Stack

Frontend

Technology	Version	Purpose
React	19.x	UI framework
TypeScript	5.x	Type-safe JavaScript
Vite	7.x	Build tool and dev server
React Router DOM	7.x	Client-side routing
Recharts	—	Data visualization (area, bar, pie, radar charts)
Framer Motion	—	Animations and transitions
Lucide React	—	Icon library
Axios	—	HTTP client

Backend

Technology	Version	Purpose
Python	3.9+	Runtime
FastAPI	0.115+	Web framework
Uvicorn	—	ASGI server
SQLite / PostgreSQL	—	Database (dual support via DATABASE_URL)
SQLAlchemy	2.0+	ORM + schema definitions
Alembic	1.13+	Database migrations
PyJWT	2.10+	JWT token management
bcrypt	4.2+	Password hashing
Google Generative AI	0.8+	Gemini API (optional)
PyMuPDF	1.24+	PDF parsing
python-docx	1.1+	DOCX parsing
defusedxml	0.7+	Safe XML parsing
spacy	3.7+	Named entity recognition
rapidfuzz	3.9+	Fuzzy string matching
python-dateutil	2.9+	Flexible date parsing
psycopg2-binary	2.9+	PostgreSQL adapter (optional)
httpx	0.28+	Async HTTP client

Technology	Version	Purpose
beautifulsoup4	4.12+	HTML parsing
pytest	8.0+	Test framework

AI Providers

Provider	Config	Notes
Google Gemini	GEMINI_API_KEY	Primary: analysis, roadmap generation, interview questions
OpenAI-compatible	OPENAI_API_KEY, OPENAI_BASE_URL, OPENAI_MODEL	Works with OpenAI, Groq, Together, OpenRouter, or local Ollama
Local fallback	none	Enhanced regex + spaCy NER parser (no LLM required)

4. Project Structure

The repository is organised as a monorepo: a FastAPI backend under `api/`, a React frontend under `frontend/`, and Alembic migrations at the root. Route handlers are split by concern, and the service layer is fully decoupled from the HTTP layer.

```

skillpath.ai/
├── api/
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   ├── db_compat.py
│   ├── auth.py
│   ├── security.py
│   ├── exceptions.py
│   ├── extractor.py
│   ├── resume_parser.py
│   ├── parser_enhancements.py
│   ├── resume_patterns.py
│   ├── career_services.py
│   ├── skill_matching.py
│   ├── job_hunt_services.py
│   ├── roadmap_services.py
│   ├── ai_provider.py
│   ├── local_llm.py
│   ├── email_service.py
│   ├── mock_interview.py
│   ├── mock_interview_ai.py
│   ├── course_scraper.py
│   ├── scraper.py
│   ├── market_data.py
│   ├── courses.py
│   ├── course_data.py
│   ├── seed_data.py
│   ├── seeders.py
│   ├── seed_content.py
│   ├── seed_defaults.py
│   ├── skills_taxonomy.py
│   ├── i18n.py
│   ├── trends.py
│   ├── requirements.txt
│   └── routes/
│       ├── admin.py
│       ├── auth.py
│       ├── user.py
│       ├── analysis.py
│       ├── jobs.py
│       ├── sharing.py
│       ├── misc.py
│       └── health.py
│   ├── data/
│   │   └── mock_questions.json
│   └── tests/
│       ├── conftest.py
│       └── test_integration.py
│   # Backend (FastAPI)
│   # App entry, middleware, CORS, router registration
│   # DB connections, init, migrations entry
│   # SQLAlchemy table definitions (38 tables)
│   # SQLite/PostgreSQL compatibility layer
│   # JWT creation/validation, auth dependencies
│   # Password hashing (bcrypt)
│   # Custom exceptions
│   # Resume text extraction (PDF/DOCX/Gemini)
│   # Local fallback resume parsing
│   # spaCy + rapidfuzz + dateutil enhancements
│   # Regex/keyword patterns for parsing
│   # Resume scoring, skill analysis
│   # Skill matching helpers
│   # Job matching, JD comparison
│   # Roadmap generation logic
│   # Gemini + OpenAI-compatible provider chain
│   # [REMOVED] Local Qwen2 model integration
│   # SMTP delivery (OTP, password reset)
│   # Static interview questions
│   # AI-generated interview sessions
│   # Coursera scraping, Udemy fallback
│   # Market data simulation
│   # Market trend data + cache
│   # Course recommendation logic
│   # Course/roadmap/field data
│   # In-memory caches + loaders/getters
│   # DB seeding functions
│   # Default seed content
│   # Default roles/roadmaps data
│   # Skill taxonomy data
│   # Translation support
│   # Trend analysis helpers
│   # Python dependencies
├── routes/
│   ├── admin.py
│   ├── auth.py
│   ├── user.py
│   ├── analysis.py
│   ├── jobs.py
│   ├── sharing.py
│   ├── misc.py
│   └── health.py
│   # Admin CRUD, audit, export, taxonomy
│   # Register, login, logout, refresh, password
│   # Profile, preferences, history, analysis
│   # Resume analysis pipeline
│   # Job matches, interview, JD compare
│   # Public report sharing
│   # Feedback, billing, notifications, i18n
│   # Health check
├── data/
│   └── mock_questions.json
│   # Static interview questions
├── tests/
│   ├── conftest.py
│   └── test_integration.py
│   # Backend test suite (pytest)
│   # Test env isolation (temp DB, no SMTP)
│   # API integration tests

```

```

├── test_features.py      # Unit tests for core services
├── test_security_fixes.py # Security regression tests
├── frontend/            # Frontend (React + Vite)
│   ├── src/
│   │   ├── App.tsx      # Routes (lazy-loaded)
│   │   ├── main.tsx     # Entry point
│   │   ├── types/
│   │   │   └── index.ts # Shared TypeScript interfaces
│   │   ├── pages/       # Landing, Analyzer, AnalysisResult, Admin, ...
│   │   ├── components/  # Feature-scoped component modules
│   │   │   ├── analyzer/ # Resume upload + analysis UI
│   │   │   ├── results/  # Analysis results (score, gaps, roadmap)
│   │   │   ├── sidebar/  # App shell navigation
│   │   │   ├── trends/   # Market trend charts
│   │   │   ├── auth/     # Login/register modal
│   │   │   ├── admin/    # Admin panels
│   │   │   ├── landing/  # Landing page sections
│   │   │   ├── interview/ # Mock interview UI
│   │   │   ├── profile/  # Profile + skill trends
│   │   │   ├── settings/ # Account settings
│   │   │   └── analysis/  # Shared analysis components
│   │   ├── context/
│   │   │   └── AuthContext.tsx # Auth state management
│   │   ├── services/
│   │   │   ├── api.ts     # Axios instance + interceptors
│   │   │   └── env.ts     # Environment config
│   │   └── styles/
│   │       ├── theme.css  # Design tokens + global styles
│   │       └── animations.css # Keyframe animations
│   ├── tsconfig.json
│   └── package.json
├── alembic/             # Database migrations
│   ├── env.py           # Migration environment config
│   ├── script.py.mako   # Migration template
│   └── versions/
│       └── 001_initial_schema.py # Initial schema migration
├── alembic.ini          # Alembic configuration
├── docker-compose.yml   # PostgreSQL service for local development
├── pytest.ini           # Pytest configuration
├── package.json         # Root scripts (setup, dev, build)
├── .env.example         # Environment variable template
└── venvapp/            # Python virtual environment (local only)

```

5. Database Schema

Dual Database Support

- **SQLite** (`api/cv.db`) — Default for local development
- **PostgreSQL** — Production via `DATABASE_URL` env var
- **Alembic** migrations auto-applied on boot
- **Compatibility layer** (`db_compat.py`) handles SQL dialect differences

All 38 Tables

The schema spans authentication, resume-analysis history, the career-role taxonomy, market-trend caching, and platform administration. Every table is defined in `api/models.py` using SQLAlchemy 2.0 declarative models.

Table	Purpose	Key Columns
users	User accounts	username, email, hashed_password, role, is_active, email_verified
user_data	Resume analysis history	analysis_data (JSON), resume_score, target_role, content_hash
user_feedback	User feedback	feed_name, feed_email, feed_score (1-5), comments
user_profiles	Extended profile	full_name, phone, location, bio, linkedin_url, github_url
user_preferences	Career preferences	target_role, timeline_months, salary_target, locale
refresh_tokens	JWT refresh tokens	token (SHA-256 hash), user_id, expires_at
otp_codes	Email OTPs	email, purpose, code_hash, expires_at, attempts, used
password_reset_tokens	Password reset	token, user_id, expires_at, used
login_attempts	Failed login tracking	username, attempts, first_attempt, locked_until
analysis_cache	Analysis cache	(content_hash, target_role) composite PK, result_json, expires_at
rate_limits	Rate limit buckets	key, count, updated_at
request_logs	Request logging	request_id, method, path, status_code, elapsed_ms
courses	Course recommendations	field, course_name, course_url, platform, rating, duration
job_roles	Job roles	title, description, category, is_active
job_role_skills	Skills per role	job_role_id, skill_name, is_required
career_roadmaps	Role roadmaps	job_role_id, title, description, duration_weeks
roadmap_steps	Roadmap steps	roadmap_id, step_number, title, skills, resources
skill_categories	Skill taxonomy	name, description
skills	Individual skills	category_id, name
role_synonyms	Role → category	role_fragment, category_name
skill_aliases	Skill aliases	alias, canonical_name

Table	Purpose	Key Columns
field_keywords	Field prediction	field_name, keyword, weight
industry_trends	Market trends	field_name, trend_type, data (JSON)
market_role_aliases	Role → field	role_name, field_name
skill_recommendations	Skill recs	field_name, skill_name, priority
roadmap_templates	Roadmap templates	field_name, template_data (JSON)
learning_actions	Learning actions	skill_name, action_text
learning_resources	Learning resources	skill_name, resource_type, title, url
skill_difficulty	Skill difficulty	skill_name, difficulty (1-3)
skill_clusters	Skill clusters	cluster_name, skills (JSON)
video_resources	Video resources	skill_name, video_type, title, url
role_configs	Role config	role_name, config_data (JSON)
market_trends_cache	Market data cache	field, source, payload, fetched_at
shared_reports	Share links	token, user_id, analysis_id, expires_at, is_public
notifications	Notifications	user_id, channel, message, status, send_at
subscriptions	Billing	user_id, plan, status, renews_at
audit_logs	Admin audit	admin_user_id, action, target_type, details, ip_address
user_roadmap_progress	Roadmap progress	user_id, analysis_id, phase_index, task_index, completed

6. Resume Parsing Pipeline

6.1 — File Upload & Validation

Endpoint: `POST /api/analyze`

Validation Steps

1. File size check (max 5MB)
2. Magic byte validation — PDF: `%PDF` (hex `25 50 44 46`); DOCX: `PK\x03\x04` (hex `50 4B 03 04`)
3. Extension is NOT trusted — only magic bytes determine file type

Code location

`api/routes/analysis.py:140-165`

6.2 — Text Extraction

PDF Extraction — `api/extractor.py:14-22`

```
doc = fitz.open(pdf_path)
text = ""
for page in doc:
    text += page.get_text() + "\n"
```

DOCX Extraction — `api/extractor.py:27-38`

```
doc = docx.Document(docx_path)
text = ""
for para in doc.paragraphs:
    if para.text:
        text += para.text + "\n"
```

6.3 — Local Regex Parser (Tier 1)

File: `api/resume_parser.py` (499 lines). This tier **always runs first** — it produces a confidence score that determines whether the AI tier is invoked.

1 • Section Detection — `_detect_section`

- Strips leading bullets (•, ·, -, *, ▪) and trailing punctuation
- Word-boundary matches against 40+ header variants
- Canonical order: summary → experience → education → skills → projects → certifications → languages → awards
- Max header length: 40 characters

Section Header Variants

```
SECTION_HEADERS = {
    "experience": ["experience", "work experience", "professional experience",
                  "work history", "employment", "employment history",
                  "professional background", "career history", "relevant experience"],
    "education": ["education", "academic", "academic background",
                  "qualifications", "education & training", "university"],
    "skills": ["skills", "technical skills", "technical stack",
              "core competencies", "competencies", "technologies",
              "skills & tools", "key skills", "areas of expertise"],
    "projects": ["projects", "personal projects", "academic projects",
                "portfolio", "selected projects", "notable projects"],
    "summary": ["summary", "profile", "objective", "about me",
               "professional summary", "career objective", "about", "overview"],
    "certifications": ["certifications", "certificates", "licenses",
                      "professional certifications", "credentials"],
    "languages": ["languages", "language skills", "spoken languages"],
    "awards": ["awards", "honors", "achievements", "recognition", "publications"],
}
```

2 • Entity Extraction

- Email — `[\w.+-]+@[\w.-]+\.\w+`
- LinkedIn — `linkedin\.\com/in/[\w-]+`
- GitHub — `github\.\com/[\w-]+`
- Phone — 10-15 digits, prefers `+` or parens

3 • Skill Extraction — `extract_skills_fuzzy` in `parser_enhancements.py`

- Uses `rapidfuzz` with `fuzz.WRatio` scorer
- Threshold: 85% similarity
- Generates 1-3 grams from text tokens
- Matches against full skill taxonomy
- Catches variants: "JS" → "JavaScript", "k8s" → "Kubernetes"

4 • Skill Inference — `infer_implicit_skills`

- Expands found skills using the inference graph
- Example: "React" → ["javascript", "html", "css", "jsx", "typescript"]
- 60+ technology mappings across all domains

5 • Name Detection — `_detect_name`

- Primary: spaCy NER (PERSON entity) on first 10 lines
- Fallback: 2-4 Title-Case words, no digits, no noise

6 • Experience Parsing — `_parse_experience_blocks`

- Date-range detection using regex + dateutil
- Scans backwards from date line for title/company headers
- Scans forwards for bullet points
- Stops bullets at next title-like line

7 • Education Parsing — `_parse_education_blocks`

- Degree keyword matching (PhD, MBA, MS, BS, etc.)
- Year extraction: `\b(?:19|20)\d{2}\b`
- Institution = remaining text after removing degree/year

8 • Company Extraction — `extract_companies_spacy`

- spaCy ORG entities
- Filters out false positives (university, college, etc.)

9 • Match Score — `_compute_local_match_score`

```
score = 20 + (required_matched × 12) + (nice_matched × 5)
range: 20-95
```

10 • Confidence Scoring

```
+20 for name found
+20 for email found
+5  for phone found
+25 for 5+ skills found
+15 for experience blocks found
+10 for education blocks found
+5  for summary found
max: 100
```

11 • Roadmap Generation — `generate_personalized_roadmap`

- Groups missing skills into phases
- Estimates duration based on difficulty
- Generates action items per skill
- Adds career prep phase

6.4 — AI Provider Chain (Tier 2)

File: `api/ai_provider.py`

Provider order (from `AI_PROVIDERS` env, default `gemini,openai`):

1. **GeminiProvider** (`gemini-2.0-flash` model) — uses `google.generativeai`, structured JSON output with schema enforcement, temperature 0.1
2. **OpenAICompatibleProvider** — uses `httpx` for HTTP calls, works with any OpenAI-API-compatible endpoint, configurable model and base URL

Prompt Construction — `extractor.py:57-80`

- Target role instruction
- Strict JSON schema enforcement
- Requests: name, email, skills, education, experience, experience_blocks, education_blocks, missing_skills, match_score, roadmap

Fallback behaviour

If both AI providers fail or return an empty payload, the pipeline silently falls back to the local regex result — the user never sees an error.

6.5 — Caching

- **Cache key:** `(content_hash, target_role)` where `content_hash` is a SHA-256 of the file content and `target_role` is the requested role (or None)
- **TTL:** 7 days
- **Storage:** `analysis_cache` table
- **Invalidation:** automatic on expiry, manual via the admin panel

7. Scoring & Matching Algorithms

7.1 — Resume Score Breakdown

Total: 100 points, distributed across five weighted categories.

Category	Max Points	Criteria
Summary	15	Present if >30 chars
Education	15	Present if any education found
Experience	35	Entry count (8) + bullet count (8) + avg bullets (6) + action verbs (6) + metrics (7)
Skills	25	Role-aware: required skills count 2x
Contact Info	10	Email (5) + phone (5)

Experience Scoring Detail

```
# Entry count (max 8)
if entries >= 3: 8 pts
elif entries == 2: 6 pts
elif entries == 1: 3 pts

# Bullet count (max 8)
if bullets >= 6: 8 pts
elif bullets >= 3: 6 pts
elif bullets >= 1: 3 pts

# Avg bullets per entry (max 6)
if avg >= 3: 6 pts
elif avg >= 2: 4 pts
elif avg >= 1: 2 pts

# Action verbs (max 6)
if verb_count >= 5: 6 pts
elif verb_count >= 3: 4 pts
elif verb_count >= 1: 2 pts

# Metrics/quantified impact (max 7)
if metric_count >= 3: 7 pts
elif metric_count >= 2: 5 pts
elif metric_count >= 1: 3 pts
```

Skills Scoring

- Required skills matched: 2 points each
- Nice-to-have skills matched: 1 point each
- Max: 25 points

7.2 — Match Score

```
match_score = 20 + (required_matched × 12) + (nice_matched × 5)  
range: 20-95
```

7.3 — Job Candidate Ranking

```
rank_score = (resume_score × 0.5) + (match_score × 0.45) - (missing_count × 1.5)
```

7.4 — Skill Gap Prioritization

Three factors determine which missing skills are surfaced first:

1. **Category count** — skills that appear in more role categories score higher
2. **Foundational bonus** — skills that are prerequisites for other missing skills
3. **Adjacent-skill bonus** — skills related to already-found skills

7.5 — Field Prediction

Algorithm: weighted keyword matching.

- Designation matches: 3x weight
- Exact skill matches: 2x weight
- Partial skill matches: 1x weight
- Objective/summary matches: 1x weight
- Minimum threshold: 3 points

8. Authentication & Security

8.1 — JWT Token System

Access Token

- Algorithm: HS256
- Expiry: 30 minutes
- Storage: httpOnly cookie (`skillpath_access`)
- Claims: `exp`, `type="access"`, `jti`

Refresh Token

- Algorithm: HS256
- Expiry: 30 days
- Storage: httpOnly cookie (`skillpath_refresh`)
- DB: SHA-256 hash stored in `refresh_tokens` table
- Rotation: new token issued on every refresh, old one deleted

Cookie Settings

- `httpOnly: true` (not accessible via JavaScript)
- `secure: true` in production
- `SameSite: lax` in development, `strict` in production
- `path: /`

8.2 — Password Security

- **Hashing:** bcrypt with 12 rounds
- **Truncation:** 72-byte limit enforced (bcrypt max)
- **Verification:** `bcrypt.checkpw()`

8.3 — OTP Verification

Registration Flow

1. User registers → account created with `email_verified=0`
2. 6-digit OTP generated, hashed (SHA-256), stored in `otp_codes`
3. OTP sent via SMTP email
4. User enters OTP → verified → `email_verified=1`

Password Reset Flow

1. User requests reset → 6-digit OTP sent to email
2. User enters OTP → verified → `reset_token` issued
3. User sets new password with `reset_token`

OTP Constraints

- Length: 6 digits
- TTL: 10 minutes
- Max attempts: 5
- Resend cooldown: 60s (prod) / 5s (dev)

8.4 — Login Lockout

- Max attempts: 5
- Lockout duration: 15 min (prod) / disabled (dev)
- Tracked in `login_attempts` table

8.5 — Rate Limiting

Global Rate Limit

- Production: 120 requests/minute/IP
- Development: 1000 requests/minute/IP
- Bucket key: `ip:minute`

Auth Endpoint Rate Limit

- Production: 20 requests/minute/IP
- Development: 1000 requests/minute/IP

Cleanup

Every 5 minutes, expired rate-limit buckets are deleted.

8.6 — Security Headers

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
Content-Security-Policy: default-src 'self'; script-src 'self';
  style-src 'self' 'unsafe-inline'; img-src 'self' data: https;;
  font-src 'self' data;; connect-src 'self' https;;
  frame-ancestors 'none'; base-uri 'self';
```

8.7 — CORS

- Configurable via `CORS_ORIGINS` env var
- Default: `http://localhost:5173,http://127.0.0.1:5173`

- Wildcard  prohibited in production
- Credentials allowed (for cookies)

9. API Reference

All endpoints are prefixed under `/api`. The **Auth** column indicates whether a valid httpOnly session cookie, an optional session, or admin privileges are required.

Authentication Endpoints

Method	Endpoint	Description	Auth
POST	/api/auth/register	Create account (sends OTP)	No
POST	/api/auth/verify-email	Verify email with OTP	No
POST	/api/auth/resend-otp	Resend verification OTP	No
POST	/api/auth/resend-verification	Resend from login screen	No
GET	/api/auth/check-username/{username}	Check username availability	No
POST	/api/auth/login	Login (username or email)	No
POST	/api/auth/logout	Logout	Yes
GET	/api/auth/me	Get current user	Cookie
POST	/api/auth/refresh	Rotate refresh token	Cookie
POST	/api/auth/request-password-reset	Request reset OTP	No
POST	/api/auth/verify-reset-otp	Verify reset OTP	No
POST	/api/auth/reset-password	Set new password	No
POST	/api/auth/change-password	Change password	Yes

Analysis Endpoints

Method	Endpoint	Description	Auth
POST	/api/analyze	Upload & analyze resume	Yes
GET	/api/user/latest-analysis	Get latest analysis	Yes
GET	/api/user/history	Get analysis history	Yes
DELETE	/api/user/history	Delete all history	Yes
DELETE	/api/user/analysis/{id}	Delete single analysis	Yes

Career Toolkit Endpoints

Method	Endpoint	Description	Auth
POST	/api/jobs/matches	Job match suggestions	Optional
POST	/api/interview/copilot	Interview questions	Optional

Method	Endpoint	Description	Auth
POST	/api/interview/simulate	AI interview turn	Optional
POST	/api/rewrite-resume	Rewrite resume bullets	Optional
POST	/api/jd/compare	Compare resume to JD	Optional
POST	/api/projects/recommend	Project recommendations	Optional
POST	/api/team/rank-candidates	Rank candidates	Optional

Mock Interview Endpoints

Method	Endpoint	Description	Auth
GET	/api/mock-interview	List roles	No
GET	/api/mock-interview/{role}	Get questions	No
POST	/api/mock-interview/start	Start AI interview	Yes
POST	/api/mock-interview/answer	Submit answer	Yes
GET	/api/mock-interview/session/{id}	Get session	Yes
POST	/api/mock-interview/finish/{id}	End session	Yes

User Endpoints

Method	Endpoint	Description	Auth
GET/PUT	/api/user/profile	Get/update profile	Yes
GET/PUT	/api/user/preferences	Get/update preferences	Yes
GET	/api/user/skill-trends	Skill trends	Yes
GET/PUT	/api/user/roadmap-progress	Roadmap progress	Yes
DELETE	/api/user/account	Delete account	Yes
POST	/api/user/contact-support	Support request	Yes

Sharing Endpoints

Method	Endpoint	Description	Auth
POST	/api/reports/share	Create share link	Yes
GET	/api/reports/share/{token}	View shared report	No
DELETE	/api/reports/share/{token}	Revoke share	Yes
GET	/api/reports/my-shares	List my shares	Yes

Admin Endpoints

Method	Endpoint	Description	Auth
GET	/api/admin/users	Resume logs	Admin
GET	/api/admin/users/{id}	Analysis detail	Admin
DELETE	/api/admin/users/{id}	Delete user data	Admin
GET	/api/admin/registered-users	List users	Admin
PATCH	/api/admin/registered-users/{id}/role	Change role	Admin
PATCH	/api/admin/registered-users/{id}/status	Activate/deactivate	Admin
DELETE	/api/admin/registered-users/{id}	Delete user	Admin
GET/POST/PATCH/DELETE	/api/admin/courses	Course CRUD	Admin
GET/POST/PATCH/DELETE	/api/admin/job-roles	Job role CRUD	Admin
GET/POST/DELETE	/api/admin/job-roles/{id}/roadmaps	Roadmap CRUD	Admin
GET	/api/admin/feedback	Feedback list	Admin
GET	/api/admin/feedback/stats	Feedback stats	Admin
DELETE	/api/admin/feedback/{id}	Delete feedback	Admin

Method	Endpoint	Description	Auth
GET	/api/admin/analytics	Platform analytics	Admin
GET	/api/admin/quality-metrics	Quality metrics	Admin
GET	/api/admin/audit-logs	Audit trail	Admin
GET/DELETE	/api/admin/analysis-cache	Cache management	Admin
GET	/api/admin/api-usage	API usage stats	Admin
GET	/api/admin/export/{table}	CSV export	Admin
POST	/api/admin/scrape-courses	Scrape courses	Admin
POST	/api/admin/trigger-scrape	Trigger market scrape	Admin

Public Endpoints

Method	Endpoint	Description	Auth
GET	/api/health	Health check	No
GET	/api/job-roles	List active roles	No
GET	/api/trends/status	Scraper status	No
POST	/api/feedback	Submit feedback	Yes
GET	/api/billing/plans	Billing plans	No
GET	/api/notifications	List notifications	Yes

10. Frontend Architecture

10.1 — Technology Stack

- **Framework:** React 19 with TypeScript
- **Build Tool:** Vite 7
- **Routing:** React Router DOM 7 (lazy-loaded routes)
- **State:** React Context (AuthContext) + local component state
- **HTTP Client:** Axios with interceptors
- **Charts:** Recharts
- **Icons:** Lucide React
- **Animations:** Framer Motion

10.2 — Routing Structure

```
/                → Landing (public)
/shared/:token   → SharedReport (public)
/app            → Analyzer (authenticated)
/analysis       → AnalysisResult (authenticated)
/settings       → Settings (authenticated)
/profile        → Profile (authenticated)
/admin         → Admin Dashboard (admin only)
/admin/dashboard → Admin Dashboard
/admin/resumes  → Resume Logs
/admin/users    → User Management
/admin/feedback → Feedback Management
/admin/courses  → Course Management
/admin/job-roles → Job Role Management
/admin/ai-monitoring → AI Monitoring
/mock-interview → Mock Interview (authenticated)
```

10.3 — Authentication Flow

AuthContext — context/AuthContext.tsx

- Reads `/api/auth/me` on mount
- Caches profile in localStorage (`skillpath_profile`)
- Provides: `user`, `loading`, `login()`, `logout()`, `updateUser()`
- Axios interceptor handles 401 → redirects to login

Login Flow

1. User submits credentials → POST `/api/auth/login`
2. Server sets httpOnly cookies (access + refresh)
3. Client calls `/api/auth/me` to get user profile

4. Profile cached in localStorage

Token Refresh

- Access token expires after 30 min
- Refresh token valid for 30 days
- POST `/api/auth/refresh` rotates refresh token
- Automatic via cookie (no client-side token management)

10.4 — Key Components

AuthModal — components/AuthModal.tsx

- Login / Register tabs
- Email OTP verification flow
- Password reset flow
- "Username or email" login field

Sidebar — components/Sidebar.tsx

- Collapsible (persisted to localStorage)
- Role-based sections (admin vs user)
- Navigation groups: Analyze / Track / Manage

ErrorBoundary — components/ErrorBoundary.tsx

- Class-based error boundary
- Recovery UI with reset button
- Logs errors via componentDidCatch

ProtectedRoute — components/ProtectedRoute.tsx

- Route guard with role-based access
- `allowedRoles` / `excludedRoles` props

10.5 — State Management

Local Storage Keys

- `skillpath_profile` — Cached user profile
- `skillpath_sidebar_collapsed` — Sidebar state

API Service — services/api.ts

- Axios instance with `withCredentials: true`
- Response interceptor: 401 → logout handler
- `registerLogoutHandler(fn)` for custom 401 handling

11. Configuration & Environment

11.1 — Required Environment Variables

```
# Required for AI features
GEMINI_API_KEY=your_gemini_api_key_here

# Security (required in production)
JWT_SECRET_KEY=at-least-32-random-characters-please

# Database
DB_FILE=api/cv.db

# CORS
CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173

# Environment
ENV=development
```

11.2 — Optional Environment Variables

```
# Database (PostgreSQL)
DATABASE_URL=postgresql://user:password@host:5432/dbname

# Admin account (auto-created on first boot)
ADMIN_USERNAME=admin
ADMIN_PASSWORD=change_me_in_production

# AI Provider Chain
AI_PROVIDERS=gemini,openai
OPENAI_API_KEY=your_key
OPENAI_BASE_URL=https://api.openai.com/v1
OPENAI_MODEL=gpt-4o-mini

# Email (SMTP)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=465
SMTP_USERNAME=you@gmail.com
SMTP_PASSWORD=xxxx_xxxx_xxxx_xxxx
SMTP_FROM_NAME=SkillPath
SMTP_STARTTLS=false

# Rate Limiting
RATE_LIMIT_PER_MINUTE=120

# Market Data
MARKET_DATA_PROVIDER=theirstack
THEIRSTACK_API_KEY=your_key
```

11.3 — Environment-Specific Behavior

Behavior	Development	Production
JWT Secret	Random per-process	Required from env
Rate Limit	1000/min	120/min
Auth Rate Limit	1000/min	20/min
Login Lockout	Disabled	15 min
OTP Resend Cooldown	5s	60s
Cookie Secure	false	true
Cookie SameSite	lax	strict
CORS Wildcard	Allowed	Prohibited
Admin Password	Any	Must not be default

12. Algorithms Deep Dive

12.1 — Skill Inference Graph

File: `api/parser_enhancements.py`

Concept: when a skill is found, infer related skills that the candidate likely knows.

Coverage

- Frontend frameworks (React, Angular, Vue, Svelte)
- Backend frameworks (Flask, Django, Express, Spring)
- Languages (JavaScript, Python, Java, Go, Rust)
- Databases (PostgreSQL, MongoDB, Redis)
- Cloud platforms (AWS, GCP, Azure)
- DevOps tools (Docker, Kubernetes, Terraform)
- ML/Data tools (TensorFlow, PyTorch, Spark)
- Mobile development (Flutter, React Native)
- Testing frameworks (Jest, Pytest, Selenium)
- APIs & protocols (GraphQL, REST, gRPC)

Example Mappings

```
"react" → ["javascript", "html", "css", "jsx", "typescript"]
"flask" → ["python", "rest api", "web development"]
"kubernetes" → ["containerization", "devops", "infrastructure", "orchestration"]
"tensorflow" → ["python", "machine learning", "deep learning", "neural networks"]
```

12.2 — Fuzzy Skill Matching

Library: rapidfuzz (`fuzz.WRatio`)

Process

1. Tokenize text into words (1-3 grams)
2. For each token, find best match in skill taxonomy
3. Score using Weighted Ratio (WRatio)
4. Threshold: 85% similarity
5. Title-case results for consistency

Noise Filter

- Single letters (c, r, go)
- Generic computing terms (cpu, gpu, ram)
- Protocols (tcp, udp, ip, ftp)
- Media formats (png, jpg, mp3)

12.3 — Date Parsing

Library: python-dateutil + custom regex

Supported Formats

- "Jan 2020 - Present"
- "January 2020 – December 2023"
- "2020 - 2023"
- "01/2020 - 12/2023"
- "since 2019"
- "March 2021 – current"

Process

1. Quick check: must contain date-like component (month name, year, etc.)
2. Try precompiled regex patterns (most specific first)
3. Fallback: dateutil parser on month-name-containing text
4. Normalize output: "Jan 2020", "Present"

12.4 — Experience Block Parsing

Algorithm

1. Find all lines containing date ranges
2. For each date line: scan backwards to collect header lines (non-bullet, <100 chars); scan forwards to collect bullet lines until next date or title
3. Extract title/company from headers or same-line content
4. Merge: prefer explicit headers, fall back to same-line extraction

Header Line Rules

- No bullet prefix
- Less than 100 characters
- No blank lines between headers
- 2+ headers: first = title, second = company
- 1 header: try "at/@/-" separator

Bullet Line Rules

- Starts with bullet character OR indented text
- Stops at next date line
- Stops at next title-like line (short, title case, title keyword)

12.5 — Roadmap Generation

Algorithm

1. Get missing skills for target role
2. Prioritize by: category count, foundational bonus, adjacent-skill bonus
3. Group related skills into phases (by cluster)
4. Estimate duration based on difficulty
5. Generate action items per skill
6. Add career prep phase (interview prep, portfolio)

Duration Estimation

- Easy (≤ 1.3): 1 week per skill
- Medium (≤ 2.3): 2 weeks per skill
- Hard (> 2.3): 3 weeks per skill

Phase Structure

```
{
  "step": 1,
  "title": "Fundamentals",
  "duration": "2 weeks",
  "skills": ["skill1", "skill2"],
  "action_items": ["Learn X", "Build Y", "Read Z"],
  "resources": [{"title": "...", "url": "..."}],
```

```
"difficulty": "Fundamentals"  
}
```

13. Testing

13.1 — Test Suite Overview

- **Framework:** pytest 8.0+
- **Total Tests:** 74
- **Location:** `api/tests/`

13.2 — Test Files

File	Tests	Focus
test_features.py	30	Resume parser, scoring, skill matching
test_integration.py	34	API endpoints, auth flows
test_security_fixes.py	10	Security regression tests

13.3 — Test Environment

Configuration — `conftest.py`

- Temporary SQLite database (per test session)
- SMTP disabled (no real emails)
- Gemini API key disabled
- ENV=development

Isolation

- Each test class gets fresh database state
- Rate limits cleared between tests
- Shared test client with `raise_server_exceptions=False`

13.4 — Key Test Cases

Parser Tests

- Name detection (spaCy + regex fallback)
- Experience block structure (title, company, dates, bullets)
- Education block structure (degree, institution, year)
- Skill extraction (taxonomy + fuzzy matching)
- Skill inference graph
- Section header variants
- Confidence scoring

Integration Tests

- Registration + OTP verification flow
- Login (username + email)
- Login blocked until email verified
- Password reset flow
- Rate limiting (429 responses)
- Resend OTP cooldown

Security Tests

- Password hashing/verification
- JWT token creation/validation
- Token refresh rotation
- OTP attempt limits
- Login lockout

14. Deployment

14.1 — Backend Deployment

```
# Activate virtual environment
source venvapp/bin/activate

# Install dependencies
pip install -r api/requirements.txt

# Download spaCy model
python -m spacy download en_core_web_sm

# Run migrations (auto on boot)
# Start server
uvicorn api.main:app --host 0.0.0.0 --port 8000
```

14.2 — Frontend Deployment

```
cd frontend
npm install
npm run build
# Serve dist/ with any static file server
```

14.3 — Environment Setup

Production Checklist

- Set `ENV=production`
- Set `JWT_SECRET_KEY` (32+ random chars)
- Set `DATABASE_URL` for PostgreSQL
- Set `CORS_ORIGINS` (no wildcard)
- Set `ADMIN_PASSWORD` (strong)
- Configure SMTP for email
- Set AI provider API keys
- Run `python -m spacy download en_core_web_sm`

14.4 — Docker Support

`docker-compose.yml` includes:

- PostgreSQL 16 service
- Environment variables for DB connection

```
docker compose up -d db  
DATABASE_URL=postgresql://skillpath:skillpath@localhost:5432/skillpath
```

Appendix A: Skill Taxonomy

8 Categories, 116+ Skills

Category	Count	Examples
Programming Languages	20	Python, JavaScript, Java, Go, Rust, C#
Web Frameworks	17	React, Angular, Django, Flask, Express
Databases	14	PostgreSQL, MongoDB, Redis, Cassandra
Cloud & DevOps	18	AWS, Docker, Kubernetes, Terraform
Data Science & AI	15	TensorFlow, PyTorch, Pandas, Spark
Mobile Development	8	Flutter, React Native, Swift, Kotlin
Soft Skills	10	Leadership, Communication, Teamwork
Other Technical	14	Git, GraphQL, REST, Microservices

Appendix B: Role Configurations

22 Predefined Roles

- Backend Engineer, Frontend Engineer, Full Stack Engineer
- Data Scientist, Data Analyst, Data Engineer
- DevOps Engineer, Cloud Engineer, SRE
- Mobile Developer (iOS/Android)
- ML Engineer, AI Engineer
- Security Engineer, QA Engineer
- Product Manager, Engineering Manager
- UI/UX Designer, Technical Writer

Each Role Has

- 6 required (core) skills
- 9 nice-to-have skills
- 4-phase career roadmap
- Project suggestions
- Interview focus areas

About this document

This document covers every technical aspect of the SkillPath.ai platform. For questions or updates, refer to the source code directly.